

A SMALL THEME FOR MODERN SLIDES

Linux OS & CLI Mastery

Modern lecture slides for Odoo server fluency

Weblearns Academy
weblearns@training.local
Odoo Full-Stack Developer Program
Lecture materials / July 2026



Agenda

- 1 Design the mental model: kernel, shell, services
- 2 Navigate and manage files with precision
- 3 Operate users, processes, systemd, and logs
- 4 Secure SSH, firewall, and production layout

TOC

1 A theme for modern talks

How this deck is taught

AWESOME-STYLE FRAME

Number rails, angled title art, and section slides are built in.

METROPOLIS DEFAULTS

Progress bars, block styles, and fonts are set from theme options.

DENSE TEACHING

Two-column frames pack objectives, examples, and pitfalls.

Ubuntu LTS

CLI

systemd

SSH

1 L01 – L05

Linux OS & CLI Mastery · teaching block 1

L01 OS Fundamentals and Kernel Basics

Objectives, key points, and lab target

Objectives

- Explain the relationship between hardware, kernel, system libraries, and user processes.
- Identify where the shell, services, and Odoo workers fit in the operating system model.
- Use basic inspection commands to confirm kernel and distribution details.
- Describe why kernel resources affect application performance.

Key points

- The kernel manages CPU scheduling, memory, devices, filesystems, and network stacks.
- User space contains shells, package managers, systemd, Python, PostgreSQL, Nginx, and Odoo.
- A distribution combines the Linux kernel with tools, defaults, repositories, and service...
- Applications fail when kernel limits are exhausted even if their own code is correct.
- Version and architecture details matter when installing drivers, Python wheels, and database...

Lab target

On a lab VM, record kernel, distribution, CPU, memory, and top-level process information. Write a short note explaining which layer would own a network failure, a Python import failure, and an out-of-memory crash.

L01 OS Fundamentals and Kernel Basics

Teaching notes

OS Fundamentals and Kernel Basics matters because Odoo servers are ordinary Linux systems before they are application servers. A developer who understands the kernel boundary and the role of user space can reason from first principles instead of copying commands blindly.

The practical workflow is to observe the current state, make one controlled change, and verify the result. For this lesson, that means paying attention to kernel version, distribution release, CPU architecture, memory, and process ownership and keeping the command output as evidence.

Linux rewards precision: paths, users, permissions, units, and sockets all have exact meanings. When those details are treated casually, the common failure is debugging symptoms at the application layer while ignoring resource or kernel limits, which is harder to diagnose after several unrelated changes.

Also remember

- In an Odoo deployment, Odoo workers, PostgreSQL, Nginx, and cron all compete for CPU time, memory, file descriptors,...
- Senior engineers build small repeatable checks around this area instead of relying on memory. They record the expected...

L01 OS Fundamentals and Kernel Basics

Example and pitfalls

Inspect the operating system baseline

These commands establish the kernel, distribution, CPU, and memory baseline before troubleshooting. Capture this information in support tickets because package names, service...

```
uname -a  
cat /etc/os-release  
lscpu | sed -n '1,8p'  
free -h
```

Common mistakes

- Treating Linux as a black box and jumping directly into Odoo configuration.
- Assuming two Ubuntu releases behave identically because the commands look similar.
- Ignoring kernel limits such as open files, memory pressure, and socket backlog.
- Changing production settings without first recording the current system state.

See user-space processes

```
ps -eo pid,ppid,user,stat,comm --sort=pid | head -20  
pstree -ap | head -30
```

L01 OS Fundamentals and Kernel Basics

Extra patterns

Linux layers

-

L02 Terminal Navigation with pwd, cd, ls, and tree

Objectives, key points, and lab target

Objectives

- Navigate absolute and relative paths confidently.
- Use directory listings to understand an unfamiliar server.
- Distinguish the current shell directory from service working directories.
- Build safe habits before editing production files.

Key points

- pwd prints the shell's current working directory; it does not tell you where a service runs.
- cd changes directory for the current shell session only.
- ls -la shows hidden files, permissions, owners, sizes, and timestamps.
- tree is useful for understanding project shape but should be limited with -L on large...
- Absolute paths start at /, while relative paths start from the current directory.

Lab target

Create a small directory tree under /tmp/odoo-navigation-lab, navigate it using absolute and relative paths, then produce a one-screen map with tree -L 3 and annotations for config, logs, and addons.

L02 Terminal Navigation with pwd, cd, ls, and tree

Teaching notes

Terminal Navigation with pwd, cd, ls, and tree matters because Odoo servers are ordinary Linux systems before they are application servers. A developer who understands filesystem navigation and situational awareness can reason from first principles instead of copying commands blindly.

The practical workflow is to observe the current state, make one controlled change, and verify the result. For this lesson, that means paying attention to current directory, hidden files, symlinks, ownership, and directory depth and keeping the command output as evidence.

Linux rewards precision: paths, users, permissions, units, and sockets all have exact meanings. When those details are treated casually, the common failure is editing or deleting the wrong file because the shell was in an unexpected directory, which is harder to diagnose after several unrelated changes.

Also remember

- In an Odoo deployment, Odoo source, custom addons, logs, configuration, and backups usually live in different...
- Senior engineers build small repeatable checks around this area instead of relying on memory. They record the expected...

L02 Terminal Navigation with pwd, cd, ls, and tree

Example and pitfalls

Build a safe navigation habit

This sequence confirms the starting point, moves to the expected application directory, and inspects contents before making changes. The limited tree depth gives a useful...

```
pwd
cd /opt/odoo
pwd
ls -la
tree -L 2 /opt/odoo
```

Common mistakes

- Running rm, mv, or chmod before checking pwd.
- Forgetting that hidden dotfiles affect shell and application behavior.
- Using tree without a depth limit on large production directories.
- Editing a symlink target without realizing where the real file lives.

Compare absolute and relative paths

```
cd /etc/odoo
ls -l odoo.conf
ls -l /etc/odoo/odoo.conf
cd /
ls -l etc/odoo/odoo.conf
```

L03 File Manipulation: Create, Copy, Move, Link, and Remove

Objectives, key points, and lab target

Objectives

- Create and inspect files safely from the terminal.
- Copy and move files while preserving important metadata when needed.
- Understand hard links, symbolic links, and backup naming.
- Use cautious deletion practices on servers.

Key points

- touch creates an empty file or updates timestamps.
- cp copies data; cp -a preserves mode, owner, timestamps, and symlinks where permissions allow.
- mv renames within a filesystem and moves across filesystems by copying then removing.
- ln -s creates a symbolic link, which stores a path reference rather than file content.
- rm is permanent at the shell level; production cleanup should be explicit and reviewed.

Lab target

Create a fake /tmp/odoo-file-lab config file, back it up with a timestamp, edit one setting, compare with diff, then replace a current symlink to point at a new fake release directory.

L03 File Manipulation: Create, Copy, Move, Link, and Remove

Teaching notes

File Manipulation: Create, Copy, Move, Link, and Remove matters because Odoo servers are ordinary Linux systems before they are application servers. A developer who understands safe manipulation of files and directories can reason from first principles instead of copying commands blindly.

The practical workflow is to observe the current state, make one controlled change, and verify the result. For this lesson, that means paying attention to destination paths, overwrite behavior, timestamps, ownership, and symbolic links and keeping the command output as evidence.

Linux rewards precision: paths, users, permissions, units, and sockets all have exact meanings. When those details are treated casually, the common failure is destroying the only known-good configuration during a rushed fix, which is harder to diagnose after several unrelated changes.

Also remember

- In an Odoo deployment, Odoo configuration files, custom modules, attachment filestore data, and backup archives...
- Senior engineers build small repeatable checks around this area instead of relying on memory. They record the expected...

L03 File Manipulation: Create, Copy, Move, Link, and Remove

Example and pitfalls

Back up and edit a configuration file

The archive copy preserves the original metadata and embeds a timestamp in the backup name. A diff after editing keeps the review focused on exactly what changed before the...

```
sudo cp -a /etc/odoo/odoo.conf /etc/odoo/odoo.conf.bak
sudo nano /etc/odoo/odoo.conf
sudo diff -u /etc/odoo/odoo.conf.bak.* /etc/odoo/odoo.conf
```

Common mistakes

- Copying config files without preserving permissions, then wondering why services cannot...
- Using wildcard deletes without first listing what the wildcard expands to.
- Confusing a symlink with a physical copy of a directory.
- Leaving backup files in locations where applications accidentally load them.

Use a symlink for active custom addons

```
sudo mkdir -p /opt/odoo/custom-addons/releases/2026-07-24
sudo ln -sf /opt/odoo/custom-addons/releases/2026-07-24 /opt/odoo/custom-addons/current
ls -l /opt/odoo/custom-addons/current
```

L04 Permissions with chmod, chown, and chgrp

Objectives, key points, and lab target

Objectives

- Read Unix permission bits and ownership from ls output.
- Apply chmod using symbolic and numeric modes.
- Change file owners and groups safely with chown and chgrp.
- Choose permissions suitable for Odoo configuration, source code, and logs.

Key points

- Permission triplets apply to owner, group, and others in that order.
- Execute on a directory means the user can traverse it; read means the user can list names.
- chmod 640 is common for private configuration files readable by owner and group only.
- chown changes owner and optionally group; chgrp changes only the group.
- Recursive chmod and chown are high-risk operations on production systems.

Lab target

In /tmp/odoo-permission-lab, create a config file and addon directory. Set ownership and permissions so a simulated odoo user can read them but an unrelated user cannot modify them.

L04 Permissions with chmod, chown, and chgrp

Teaching notes

Permissions with chmod, chown, and chgrp matters because Odoo servers are ordinary Linux systems before they are application servers. A developer who understands ownership and permission enforcement can reason from first principles instead of copying commands blindly.

The practical workflow is to observe the current state, make one controlled change, and verify the result. For this lesson, that means paying attention to read, write, execute bits, owner, group, others, directories, and recursive changes and keeping the command output as evidence.

Linux rewards precision: paths, users, permissions, units, and sockets all have exact meanings. When those details are treated casually, the common failure is masking an ownership problem by making sensitive files world-writable, which is harder to diagnose after several unrelated changes.

Also remember

- In an Odoo deployment, Odoo should run as a dedicated user that can read its code and config but does not own...
- Senior engineers build small repeatable checks around this area instead of relying on memory. They record the expected...

L04 Permissions with chmod, chown, and chgrp

Example and pitfalls

Set safe Odoo configuration permissions

The root user owns the file so the service cannot silently rewrite it, while the odoo group can read it. Testing as the odoo user is more reliable than assuming permissions are...

```
sudo chown root:odoo /etc/odoo/odoo.conf
sudo chmod 640 /etc/odoo/odoo.conf
ls -l /etc/odoo/odoo.conf
sudo -u odoo test -r /etc/odoo/odoo.conf && echo "odoo can read it"
```

Common mistakes

- Using chmod 777 as a troubleshooting shortcut and leaving it in place.
- Forgetting execute permission on parent directories.
- Changing ownership of system directories recursively from the wrong path.
- Testing access as root instead of the service user.

Repair ownership of a custom addon

```
sudo chown -R odoo:odoo /opt/odoo/custom-addons/my_module
find /opt/odoo/custom-addons/my_module -type d -exec chown odoo:odoo {} \;
find /opt/odoo/custom-addons/my_module -type f -exec chown odoo:odoo {} \;
```

L04 Permissions with chmod, chown, and chgrp

Extra patterns

Production rule

If the fix requires chmod 777, the real problem has not been understood yet.

L05 Users, Groups, and sudo

Objectives, key points, and lab target

Objectives

- Explain why services should run under dedicated users.
- Inspect users, groups, and sudo privileges.
- Use sudo intentionally without hiding ownership problems.
- Separate human admin access from application runtime access.

Key points

- Each process runs with a user ID and group IDs that determine access checks.
- A service account should own only the files it must manage.
- sudo records administrative intent and can restrict which users may run privileged commands.
- Group membership is a clean way to grant shared read or write access.
- Human users should log in individually instead of sharing one admin password.

Lab target

Create a lab service user, place it in a project group, and demonstrate a file that the group can read but cannot write. Then run a Python version check as that service user.

L05 Users, Groups, and sudo

Teaching notes

Users, Groups, and sudo matters because Odoo servers are ordinary Linux systems before they are application servers. A developer who understands identity and privilege separation can reason from first principles instead of copying commands blindly.

The practical workflow is to observe the current state, make one controlled change, and verify the result. For this lesson, that means paying attention to user IDs, group memberships, service accounts, sudo rules, and command auditing and keeping the command output as evidence.

Linux rewards precision: paths, users, permissions, units, and sockets all have exact meanings. When those details are treated casually, the common failure is running application processes as root and widening the blast radius of a compromise, which is harder to diagnose after several unrelated changes.

Also remember

- In an Odoo deployment, Odoo service users need predictable file access while administrators need controlled elevation...
- Senior engineers build small repeatable checks around this area instead of relying on memory. They record the expected...

L05 Users, Groups, and sudo

Example and pitfalls

Inspect identity and group membership

These commands answer who you are, who the service user is, and what privileged commands are allowed. The output is essential when a file is readable in your shell but not by the...

```
id
id odoo
getent passwd odoo
getent group odoo
sudo -l
```

Common mistakes

- Running Odoo manually with sudo and creating root-owned cache files.
- Sharing one SSH account among administrators.
- Adding broad passwordless sudo rules without command boundaries.
- Changing service file ownership instead of fixing group membership.

Run a check as the service user

```
sudo -u odoo -H bash -lc 'whoami; pwd; python3 --version'
sudo -u odoo -H test -x /opt/odoo/venv/bin/python && echo
```

2 L06 – L10

Linux OS & CLI Mastery · teaching block 2

L06 Processes with top, htop, ps, and kill

Objectives, key points, and lab target

Objectives

- List processes and interpret PID, parent PID, state, CPU, and memory columns.
- Use top or htop to observe live server pressure.
- Terminate processes with the least disruptive signal first.
- Connect process behavior to Odoo worker and cron performance.

Key points

- ps gives a point-in-time process list; top and htop show live changes.
- PID identifies a process instance, while command names may repeat many times.
- SIGTERM asks a process to stop cleanly; SIGKILL stops it immediately and prevents cleanup.
- High CPU, high memory, and uninterruptible IO wait suggest different root causes.
- Parent PID helps distinguish systemd-managed services from manual shells.

Lab target

Start a harmless sleep process, find it with ps and pgrep, send TERM, confirm it exited, then repeat with a Python process that prints its PID and handles KeyboardInterrupt.

L06 Processes with top, htop, ps, and kill

Teaching notes

Processes with top, htop, ps, and kill matters because Odoo servers are ordinary Linux systems before they are application servers. A developer who understands runtime process observation and control can reason from first principles instead of copying commands blindly.

The practical workflow is to observe the current state, make one controlled change, and verify the result. For this lesson, that means paying attention to PIDs, process states, parent-child relationships, CPU use, RSS memory, and signals and keeping the command output as evidence.

Linux rewards precision: paths, users, permissions, units, and sockets all have exact meanings. When those details are treated casually, the common failure is killing the wrong worker or masking the underlying cause of resource exhaustion, which is harder to diagnose after several unrelated changes.

Also remember

- In an Odoo deployment, Odoo may run multiple HTTP workers, longpolling workers, cron workers, PostgreSQL sessions, and...
- Senior engineers build small repeatable checks around this area instead of relying on memory. They record the expected...

L06 Processes with top, htop, ps, and kill

Example and pitfalls

Find Odoo-related processes

Sorting by memory and filtering by the odoo user quickly reveals whether application workers dominate the server. Elapsed time helps separate a fresh runaway process from a...

```
ps -eo pid,ppid,user,stat,%cpu,%mem,etime,cmd --sort=-  
pgrep -a -u odoo python  
top -o %MEM
```

Common mistakes

- Using kill -9 as the first response.
- Matching processes by a vague name and terminating unrelated commands.
- Ignoring parent processes that will immediately restart the child.
- Confusing memory percent with actual RSS on large-memory servers.

Signal a process deliberately

```
sudo kill -TERM 12345  
sleep 3  
ps -p 12345 -o pid,stat,cmd  
sudo kill -KILL 12345 # only if the process ignored TERM
```

L07 systemd Services

Objectives, key points, and lab target

Objectives

- Explain systemd units, service states, and restart policies.
- Inspect and manage Odoo service lifecycle with systemctl.
- Understand unit files, drop-ins, and daemon reloads.
- Use systemd status output as a first troubleshooting artifact.

Key points

- A service unit describes how a daemon starts, stops, reloads, and restarts.
- systemctl status combines state, main PID, recent logs, and exit code context.
- After editing unit files or drop-ins, run systemctl daemon-reload.
- Restart policies should match the failure mode and avoid tight crash loops.
- EnvironmentFile and WorkingDirectory affect how Python and Odoo locate resources.

Lab target

Inspect a sample odoo.service unit, identify ExecStart, User, Group, WorkingDirectory, and EnvironmentFile, then create a hypothetical drop-in that changes only the restart policy.

L07 systemd Services

Teaching notes

systemd Services matters because Odoo servers are ordinary Linux systems before they are application servers. A developer who understands service supervision and repeatable startup can reason from first principles instead of copying commands blindly.

The practical workflow is to observe the current state, make one controlled change, and verify the result. For this lesson, that means paying attention to unit names, active state, main PID, ExecStart, environment, dependencies, and restart behavior and keeping the command output as evidence.

Linux rewards precision: paths, users, permissions, units, and sockets all have exact meanings. When those details are treated casually, the common failure is starting Odoo manually and leaving production outside the service manager, which is harder to diagnose after several unrelated changes.

Also remember

- In an Odoo deployment, systemd keeps Odoo, PostgreSQL, and Nginx predictable across reboots and failures connects this...
- Senior engineers build small repeatable checks around this area instead of relying on memory. They record the expected...

L07 systemd Services

Example and pitfalls

Inspect and restart an Odoo service

status answers whether the service is active and shows the immediate failure context if it is not. cat displays the effective unit content, including drop-ins, which prevents...

```
sudo systemctl status odoo --no-pager
sudo systemctl cat odoo
sudo systemctl restart odoo
sudo systemctl is-active odoo
```

Common mistakes

- Editing a unit file and restarting without daemon-reload.
- Debugging an old service name while the active unit has a different name.
- Running Odoo manually and assuming systemd uses the same environment.
- Setting Restart=always without understanding a crash loop.

Create a safe override directory

```
sudo systemctl edit odoo
# Add only the changed settings in the editor, for example
# [Service]
# Restart=on-failure
sudo systemctl daemon-reload
sudo systemctl restart odoo
```

L08 journalctl and Log Files

Objectives, key points, and lab target

Objectives

- Read systemd journal entries for a service.
- Filter logs by unit, time range, priority, and boot.
- Combine journal output with application log files.
- Preserve useful evidence during Odoo incidents.

Key points

- journalctl -u filters logs for a systemd unit.
- The --since and --until options keep searches tied to the incident window.
- The current boot can be selected with -b, and previous boots with -b -1.
- Application logs may live in /var/log/odoo or another configured path.
- Tracebacks should be read from top context to final exception, not just the last line.

Lab target

Restart a harmless sample service or timer, collect journal entries for the last 10 minutes, identify the first and last relevant lines, and write a five-line incident timeline.

L08 journalctl and Log Files

Teaching notes

journalctl and Log Files matters because Odoo servers are ordinary Linux systems before they are application servers. A developer who understands log-based investigation can reason from first principles instead of copying commands blindly.

The practical workflow is to observe the current state, make one controlled change, and verify the result. For this lesson, that means paying attention to unit filters, timestamps, boot IDs, priority levels, application log paths, and rotation and keeping the command output as evidence.

Linux rewards precision: paths, users, permissions, units, and sockets all have exact meanings. When those details are treated casually, the common failure is losing the first failure message by only looking at the last repeated error, which is harder to diagnose after several unrelated changes.

Also remember

- In an Odoo deployment, Odoo logs often show Python tracebacks while systemd logs show process exit, restart, and...
- Senior engineers build small repeatable checks around this area instead of relying on memory. They record the expected...

L08 journalctl and Log Files

Example and pitfalls

Read recent service logs

The first command establishes recent context, the second focuses on warnings and errors, and the third stays within the current boot. Time filters prevent unrelated old failures...

```
sudo journalctl -u odoo --since "30 minutes ago" --no-pager
sudo journalctl -u odoo -p warning..alert --since today
sudo journalctl -u odoo -b --no-pager | tail -80
```

Common mistakes

- Searching logs without a time window.
- Reading only the final traceback line and missing the real cause above it.
- Ignoring systemd restart messages when the application log is empty.
- Overwriting logs during cleanup before preserving the incident evidence.

Follow application logs during restart

```
sudo tail -F /var/log/odoo/odoo.log
# In another terminal:
sudo systemctl restart odoo
```

L09 Editing Files with nano and vim

Objectives, key points, and lab target

Objectives

- Choose nano or vim based on task complexity and operator comfort.
- Open, edit, save, and exit safely in both editors.
- Avoid accidental configuration damage while editing as root.
- Validate edits before restarting services.

Key points

- nano is approachable for small edits and displays common shortcuts.
- vim is modal, powerful, and widely available on servers.
- Always back up important files before editing them as root.
- Use validation commands before restarting services that depend on edited config.
- sudoedit can be safer than running a full editor process as root.

Lab target

Edit a copied sample config with both nano and vim. Change one value, save it, show the diff, then intentionally abandon a second change without saving.

L09 Editing Files with nano and vim

Teaching notes

Editing Files with nano and vim matters because Odoo servers are ordinary Linux systems before they are application servers. A developer who understands terminal editing discipline can reason from first principles instead of copying commands blindly.

The practical workflow is to observe the current state, make one controlled change, and verify the result. For this lesson, that means paying attention to editor modes, save behavior, sudo usage, temporary files, syntax visibility, and post-edit validation and keeping the command output as evidence.

Linux rewards precision: paths, users, permissions, units, and sockets all have exact meanings. When those details are treated casually, the common failure is making an invisible typo in a production configuration file, which is harder to diagnose after several unrelated changes.

Also remember

- In an Odoo deployment, Odoo and Nginx configuration changes often need exact key names, indentation, and path values...
- Senior engineers build small repeatable checks around this area instead of relying on memory. They record the expected...

L09 Editing Files with nano and vim

Example and pitfalls

Safe edit sequence

sudoedit copies the file to a temporary location, opens it with your editor, and writes it back with elevated permissions. The diff provides a review checkpoint before the...

```
sudo cp -a /etc/odoo/odoo.conf /etc/odoo/odoo.conf.pre
sudoedit /etc/odoo/odoo.conf
sudo diff -u /etc/odoo/odoo.conf.pre-edit /etc/odoo/odoo.conf
```

Common mistakes

- Opening a root editor without a backup.
- Saving editor temporary files in a directory that is scanned by Odoo.
- Restarting services before checking a diff or validation command.
- Panicking in vim instead of pressing Esc and using :q! to exit without saving.

Essential vim commands

```
vim /tmp/example.conf
# i          enter insert mode
# Esc       return to normal mode
# :w        write file
# :q        quit
# :wq       write and quit
# :q!       quit without saving
```

L10 Networking with ip, ping, and ss

Objectives, key points, and lab target

Objectives

- Inspect IP addresses, routes, and listening sockets.
- Use ping for basic reachability without overinterpreting it.
- Use ss to confirm which process owns a port.
- Troubleshoot Odoo, PostgreSQL, and reverse proxy connectivity.

Key points

- ip addr shows interface addresses; ip route shows where packets will go.
- ping tests ICMP reachability, not whether TCP application ports are open.
- ss -ltnp shows listening TCP sockets and owning processes.
- A service bound to 127.0.0.1 is reachable locally but not from another host.
- PostgreSQL and Odoo ports should usually be private, with Nginx handling public HTTP.

Lab target

Draw a simple network path from browser to Nginx to Odoo to PostgreSQL. On a lab host, collect ip addr, ip route, and ss output and label which ports should be public.

L10 Networking with ip, ping, and ss

Teaching notes

Networking with ip, ping, and ss matters because Odoo servers are ordinary Linux systems before they are application servers. A developer who understands network observation from the host can reason from first principles instead of copying commands blindly.

The practical workflow is to observe the current state, make one controlled change, and verify the result. For this lesson, that means paying attention to interfaces, addresses, routes, DNS assumptions, TCP ports, process ownership, and listening bind addresses and keeping the command output as evidence.

Linux rewards precision: paths, users, permissions, units, and sockets all have exact meanings. When those details are treated casually, the common failure is assuming the application is broken when the port is not listening or traffic is routed elsewhere, which is harder to diagnose after several unrelated changes.

Also remember

- In an Odoo deployment, Odoo typically listens on localhost or a private interface while Nginx exposes HTTP and HTTPS...
- Senior engineers build small repeatable checks around this area instead of relying on memory. They record the expected...

L10 Networking with ip, ping, and ss

Example and pitfalls

Inspect addresses, routes, and ports

The commands move from host addressing to routing, reachability, and listening sockets. For Odoo, the socket list often reveals whether the service is bound only to localhost or...

```
ip addr show
ip route show
ping -c 3 8.8.8.8
ss -ltnp | grep -E ':(8069|8072|80|443|5432)\b'
```

Common mistakes

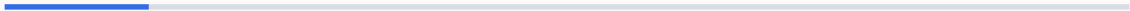
- Treating a successful ping as proof that HTTP or PostgreSQL works.
- Forgetting that localhost from one container or VM is not another host's localhost.
- Leaving Odoo bound to a public interface unnecessarily.
- Ignoring DNS and testing only raw IP addresses.

Test an Odoo port locally

```
curl -I http://127.0.0.1:8069
curl -I http://localhost:8069
ss -ltnp 'sport = :8069'
```

3 L11 – L15

Linux OS & CLI Mastery · teaching block 3



L11 SSH Keys and Hardening

Objectives, key points, and lab target

Objectives

- Explain public-key SSH authentication.
- Generate and install SSH keys safely.
- Apply basic sshd hardening for production servers.
- Avoid locking yourself out during access changes.

Key points

- The private key stays on the client; the public key is placed on the server.
- `authorized_keys` controls which public keys can log in as a user.
- `PasswordAuthentication no` reduces brute-force risk after keys are confirmed.
- `PermitRootLogin no` encourages named users and sudo auditability.
- sshd configuration should be tested before reload or restart.

Lab target

On a disposable VM, create a new SSH key, install it for a non-root user, test login, then draft a hardened sshd_config snippet with comments explaining each setting.

L11 SSH Keys and Hardening

Teaching notes

SSH Keys and Hardening matters because Odoo servers are ordinary Linux systems before they are application servers. A developer who understands secure remote administration can reason from first principles instead of copying commands blindly.

The practical workflow is to observe the current state, make one controlled change, and verify the result. For this lesson, that means paying attention to key pairs, `authorized_keys`, `sshd_config`, password login, root login, ports, and active sessions and keeping the command output as evidence.

Linux rewards precision: paths, users, permissions, units, and sockets all have exact meanings. When those details are treated casually, the common failure is weakening or breaking the only administrative access path to a production host, which is harder to diagnose after several unrelated changes.

Also remember

- In an Odoo deployment, Odoo servers need secure admin access because shell access often implies database, file, and...
- Senior engineers build small repeatable checks around this area instead of relying on memory. They record the expected...

L11 SSH Keys and Hardening

Example and pitfalls

Create and install a key

Ed25519 keys are compact and strong for modern SSH clients. The final command proves the key works before password login is disabled.

```
ssh-keygen -t ed25519 -C "admin@example.com"  
ssh-copy-id admin@odoo-server.example.com  
ssh admin@odoo-server.example.com 'whoami; hostname'
```

Common mistakes

- Disabling password login before confirming key login works.
- Closing the only active session immediately after changing sshd_config.
- Sharing one private key across administrators.
- Changing the SSH port without updating firewall or cloud rules.

Test sshd configuration before reload

```
sudo cp -a /etc/ssh/sshd_config /etc/ssh/sshd_config.pre-h  
sudoedit /etc/ssh/sshd_config  
sudo sshd -t  
sudo systemctl reload ssh
```

L12 Environment Variables and bashrc

Objectives, key points, and lab target

Objectives

- Read and set environment variables in a shell.
- Distinguish shell startup files from systemd service environments.
- Use PATH, HOME, and application variables intentionally.
- Avoid hiding production settings in interactive-only files.

Key points

- Environment variables are inherited from parent process to child process.
- export makes a shell variable available to child processes.
- .bashrc affects interactive bash shells, not all services.
- PATH controls command lookup order and can explain different Python binaries.
- systemd units can use Environment and EnvironmentFile for service-specific values.

Lab target

Create a shell variable and an exported variable, run a child Python command that prints `os.environ`, and explain why only one value is visible. Then draft a service EnvironmentFile for a sample Odoo variable.

L12 Environment Variables and bashrc

Teaching notes

Environment Variables and bashrc matters because Odoo servers are ordinary Linux systems before they are application servers. A developer who understands process environment configuration can reason from first principles instead of copying commands blindly.

The practical workflow is to observe the current state, make one controlled change, and verify the result. For this lesson, that means paying attention to exported variables, shell startup order, PATH lookup, virtual environments, and systemd EnvironmentFile usage and keeping the command output as evidence.

Linux rewards precision: paths, users, permissions, units, and sockets all have exact meanings. When those details are treated casually, the common failure is testing with variables from .bashrc that the Odoo service never receives, which is harder to diagnose after several unrelated changes.

Also remember

- In an Odoo deployment, Odoo configuration may depend on PATH, Python virtual environments, locale, addons paths, and...
- Senior engineers build small repeatable checks around this area instead of relying on memory. They record the expected...

L12 Environment Variables and bashrc

Example and pitfalls

Inspect shell environment

These commands show which executable the shell will run and which inherited values are visible. A mismatch between command -v and service ExecStart is a frequent source of...

```
echo "$PATH"
printenv | sort | grep -E '^(HOME|LANG|PATH|VIRTUAL_ENV|)'
command -v python3
python3 -c 'import sys; print(sys.executable)'
```

Common mistakes

- Adding production variables only to ~/.bashrc and expecting systemd to see them.
- Modifying PATH globally when only one service needs a specific executable.
- Storing secrets in world-readable environment files.
- Assuming sudo preserves the caller's full environment.

Use an EnvironmentFile with systemd

```
sudo install -m 640 -o root -g odoo /dev/null /etc/odoo/odoo.conf
echo 'ODOO_RC=/etc/odoo/odoo.conf' | sudo tee -a /etc/odoo/odoo.conf
sudo systemctl edit odoo
# [Service]
# EnvironmentFile=/etc/odoo/odoo.env
```

L13 Cron Jobs

Objectives, key points, and lab target

Objectives

- Understand cron timing syntax and execution environment.
- Create safe scheduled maintenance tasks.
- Redirect cron output for observability.
- Avoid conflicts between cron and Odoo's internal scheduler.

Key points

- User crontabs run as that user; `/etc/cron.d` entries include a user field.
- Cron has a small environment and often a different `PATH` from interactive shells.
- Use absolute paths for commands and files in cron jobs.
- Redirect `stdout` and `stderr` to logs or monitoring hooks.
- Locking prevents overlapping runs when a previous job is slow.

Lab target

Write a cron entry that runs a harmless script every five minutes, logs `stdout` and `stderr`, and uses `flock`. Then intentionally remove `PATH` from the script and explain the failure.

L13 Cron Jobs

Teaching notes

Cron Jobs matters because Odoo servers are ordinary Linux systems before they are application servers. A developer who understands scheduled command execution can reason from first principles instead of copying commands blindly.

The practical workflow is to observe the current state, make one controlled change, and verify the result. For this lesson, that means paying attention to minute, hour, day fields, user crontabs, system crontabs, PATH, working directory, and output handling and keeping the command output as evidence.

Linux rewards precision: paths, users, permissions, units, and sockets all have exact meanings. When those details are treated casually, the common failure is running a maintenance command with a minimal cron environment and no logs, which is harder to diagnose after several unrelated changes.

Also remember

- In an Odoo deployment, Odoo has its own scheduled actions, but server tasks such as backups, cleanup, and certificate...
- Senior engineers build small repeatable checks around this area instead of relying on memory. They record the expected...

L13 Cron Jobs

Example and pitfalls

A safe backup cron pattern

This system cron entry declares its shell and PATH, runs as postgres, prevents overlap with flock, and captures output. The command still needs a tested script and restore...

```
# /etc/cron.d/odoo-backup
SHELL=/bin/bash
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin
15 2 * * * postgres flock -n /var/lock/odoo-backup.lock /usr/local/bin/backup-odoo-db >> /var/log/odoo/backup.log
```

Common mistakes

- Using relative paths in cron commands.
- Ignoring stderr so failures disappear into local mail or nowhere.
- Scheduling backups without restore tests.
- Letting slow jobs overlap and overload the database server.

Preview a cron schedule

```
crontab -l
systemctl list-timers --all
grep -R "backup-odoo" /etc/cron* 2>/dev/null
```


L14 APT Package Management

Objectives, key points, and lab target

Objectives

- Update package metadata and install packages with apt.
- Understand repositories, package versions, and upgrades.
- Inspect installed packages and service changes.
- Apply package changes safely on Odoo servers.

Key points

- apt update refreshes metadata; it does not upgrade installed packages.
- apt install changes the system and may start or restart services.
- apt-cache policy shows candidate versions and repositories.
- apt-mark hold can prevent a sensitive package from upgrading unexpectedly.
- Security updates are important but still need operational timing.

Lab target

On a lab Ubuntu system, inspect the candidate version of nginx, run a simulated upgrade, install a small package, and record which services changed state.

L14 APT Package Management

Teaching notes

APT Package Management matters because Odoo servers are ordinary Linux systems before they are application servers. A developer who understands Debian and Ubuntu package lifecycle can reason from first principles instead of copying commands blindly.

The practical workflow is to observe the current state, make one controlled change, and verify the result. For this lesson, that means paying attention to apt update, apt install, apt upgrade, repositories, package holds, service restarts, and changelog awareness and keeping the command output as evidence.

Linux rewards precision: paths, users, permissions, units, and sockets all have exact meanings. When those details are treated casually, the common failure is upgrading core packages during business hours without understanding service impact, which is harder to diagnose after several unrelated changes.

Also remember

- In an Odoo deployment, Odoo servers depend on OS packages for Python, PostgreSQL clients, Nginx, wkhtmltopdf...
- Senior engineers build small repeatable checks around this area instead of relying on memory. They record the expected...

L14 APT Package Management

Example and pitfalls

Inspect and install a package

The policy command shows where the package will come from before installation. After installing, verify both package state and the service that the package may have started.

```
sudo apt update
apt-cache policy nginx
sudo apt install nginx
dpkg -l | grep '^ii' | grep nginx
systemctl status nginx --no-pager
```

Common mistakes

- Confusing apt update with apt upgrade.
- Installing from random third-party repositories without trust or maintenance review.
- Upgrading PostgreSQL-related packages without checking database compatibility.
- Leaving build tools installed on minimal production machines unnecessarily.

Review pending upgrades

```
sudo apt update
apt list --upgradable
sudo apt -s upgrade
```

L15 Disk and Memory Usage with df, du, and free

Objectives, key points, and lab target

Objectives

- Measure filesystem capacity with df.
- Find large directories and files with du.
- Interpret memory and swap information with free.
- Connect disk and memory pressure to Odoo failures.

Key points

- df -h shows filesystem capacity by mount point.
- df -ih shows inode usage, which can be exhausted even when space remains.
- du -sh summarizes directory size; sort helps find the largest consumers.
- free -h separates used memory from available memory and filesystem cache.
- Swap activity can indicate memory pressure but is not automatically bad.

Lab target

Fill a small lab directory with test files, use du to find the largest path, then remove only the test data. Record df output before and after and explain why the numbers changed.

L15 Disk and Memory Usage with df, du, and free

Teaching notes

Disk and Memory Usage with df, du, and free matters because Odoo servers are ordinary Linux systems before they are application servers. A developer who understands capacity and pressure diagnosis can reason from first principles instead of copying commands blindly.

The practical workflow is to observe the current state, make one controlled change, and verify the result. For this lesson, that means paying attention to mounted filesystems, inode use, directory growth, cache memory, swap use, and database storage paths and keeping the command output as evidence.

Linux rewards precision: paths, users, permissions, units, and sockets all have exact meanings. When those details are treated casually, the common failure is cleaning the wrong directory while the full filesystem remains full, which is harder to diagnose after several unrelated changes.

Also remember

- In an Odoo deployment, Odoo attachments, logs, PostgreSQL data, backups, and temporary exports can grow independently...
- Senior engineers build small repeatable checks around this area instead of relying on memory. They record the expected...

L15 Disk and Memory Usage with df, du, and free

Example and pitfalls

Find the full mount and largest directories

The -x flag keeps du on one filesystem, which prevents chasing data across mounted volumes. Checking inodes helps diagnose failures caused by too many small files, such as...

```
df -h
df -ih
sudo du -xhd1 /var | sort -h
sudo du -xhd1 /opt | sort -h
```

Common mistakes

- Deleting files from a different mount point than the one that is full.
- Ignoring inode exhaustion.
- Clearing application data without retention approval.
- Mistaking Linux filesystem cache for wasted memory.

Inspect memory pressure

```
free -h
vmstat 1 5
ps -eo pid,user,%mem,rss,cmd --sort=-rss | head -15
```

4 L16 – L18

Linux OS & CLI Mastery · teaching block 4

L16 UFW Firewall

Objectives, key points, and lab target

Objectives

- Explain host firewall allow and deny rules.
- Use UFW to expose only required services.
- Verify firewall status before and after changes.
- Design firewall policy for an Odoo server behind a reverse proxy.

Key points

- UFW is a friendly interface over Linux firewall rules.
- Default deny incoming and allow outgoing is common for application hosts.
- Rules can restrict source IPs for SSH or administrative ports.
- Nginx should usually expose 80 and 443; Odoo 8069 should stay private.
- IPv6 must be considered if the host has public IPv6 addresses.

Lab target

Design a UFW policy for a single-server Odoo deployment with Nginx, PostgreSQL local only, and SSH from a VPN range. Explain every allowed port.

L16 UFW Firewall

Teaching notes

UFW Firewall matters because Odoo servers are ordinary Linux systems before they are application servers. A developer who understands host-level network policy can reason from first principles instead of copying commands blindly.

The practical workflow is to observe the current state, make one controlled change, and verify the result. For this lesson, that means paying attention to default policies, allowed ports, source restrictions, IPv4 and IPv6, ordering, and rollback and keeping the command output as evidence.

Linux rewards precision: paths, users, permissions, units, and sockets all have exact meanings. When those details are treated casually, the common failure is locking out SSH or exposing internal Odoo and PostgreSQL ports publicly, which is harder to diagnose after several unrelated changes.

Also remember

- In an Odoo deployment, a typical Odoo host should expose SSH and HTTPS while keeping PostgreSQL and backend Odoo ports...
- Senior engineers build small repeatable checks around this area instead of relying on memory. They record the expected...

L16 UFW Firewall

Example and pitfalls

Enable a conservative Odoo firewall

This allows administration and public web traffic while denying other incoming connections. OpenSSH should be allowed before enabling UFW to avoid cutting off the active session.

```
sudo ufw status verbose
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow OpenSSH
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp
sudo ufw enable
sudo ufw status numbered
```

Common mistakes

- Enabling UFW before allowing SSH.
- Exposing PostgreSQL or Odoo backend ports to the entire internet.
- Forgetting IPv6 rules on dual-stack hosts.
- Deleting numbered rules without rechecking numbers after each deletion.

Restrict SSH to an admin network

```
sudo ufw delete allow OpenSSH
sudo ufw allow from 203.0.113.0/24 to any port 22 proto tcp
sudo ufw status numbered
```

L17 Production Directory Layout for Odoo

Objectives, key points, and lab target

Objectives

- Map common Odoo directories by responsibility.
- Separate source code, configuration, logs, data, backups, and virtual environments.
- Choose ownership and permissions for each area.
- Make deployments understandable for future maintainers.

Key points

- `/etc/odoo` is a natural place for root-owned configuration.
- `/opt/odoo` often holds application source, releases, addons, and virtual environments.
- `/var/log/odoo` should contain logs with rotation configured.
- `/var/lib/odoo` holds runtime data such as filestore when using packaged conventions.
- Backups should be isolated from live code and tested for restore.

Lab target

Design a directory layout for a production Odoo server. Include owner, group, permission mode, backup policy, and which paths are referenced by the systemd service.

L17 Production Directory Layout for Odoo

Teaching notes

Production Directory Layout for Odoo matters because Odoo servers are ordinary Linux systems before they are application servers. A developer who understands filesystem organization for maintainable deployments can reason from first principles instead of copying commands blindly.

The practical workflow is to observe the current state, make one controlled change, and verify the result. For this lesson, that means paying attention to config paths, addons paths, virtualenvs, logs, filestore, backups, releases, and ownership boundaries and keeping the command output as evidence.

Linux rewards precision: paths, users, permissions, units, and sockets all have exact meanings. When those details are treated casually, the common failure is mixing code, data, and backups until upgrades and restores become unsafe, which is harder to diagnose after several unrelated changes.

Also remember

- In an Odoo deployment, Odoo deployments require clear paths for core source, custom addons, enterprise addons, Python...
- Senior engineers build small repeatable checks around this area instead of relying on memory. They record the expected...

L17 Production Directory Layout for Odoo

Example and pitfalls

Example directory map

This layout separates configuration, source, virtual environment, runtime data, logs, and backups. The exact paths may vary, but the responsibilities should remain explicit.

```
/etc/odoo/odoo.conf
/opt/odoo/src/odoo
/opt/odoo/enterprise
/opt/odoo/custom-addons/current
/opt/odoo/venv
/var/lib/odoo/.local/share/Odoo/filestore
/var/log/odoo/odoo.log
/srv/backups/odoo
```

Common mistakes

- Storing backups inside the live addons path.
- Letting the service user own root-level configuration.
- Putting virtual environments in home directories tied to human users.
- Using undocumented paths that only one administrator understands.

Check ownership boundaries

```
sudo namei -l /etc/odoo/odoo.conf
sudo namei -l /opt/odoo/custom-addons/current
sudo -u odoo test -r /etc/odoo/odoo.conf && echo readable
sudo -u odoo test -w /etc/odoo/odoo.conf || echo not writable
```

L17 Production Directory Layout for Odoo

Extra patterns

Directory responsibilities

-

L18 Troubleshooting Checklist for Odoo Servers

Objectives, key points, and lab target

Objectives

- Apply a structured checklist during Odoo outages.
- Collect evidence before changing the server.
- Move from infrastructure checks to application checks logically.
- Write concise incident notes and next actions.

Key points

- Start by defining the user-visible symptom and scope.
- Check recent changes before assuming random failure.
- Inspect service state, logs, ports, disk, memory, and database connectivity.
- Make one change at a time and record its result.
- Prefer rollback when a recent deployment caused the incident.

Lab target

Use the checklist against a deliberately broken lab Odoo-like service. Record observations, identify the failing layer, make one fix, verify recovery, and write a brief post-incident note.

L18 Troubleshooting Checklist for Odoo Servers

Teaching notes

Troubleshooting Checklist for Odoo Servers matters because Odoo servers are ordinary Linux systems before they are application servers. A developer who understands systematic production troubleshooting can reason from first principles instead of copying commands blindly.

The practical workflow is to observe the current state, make one controlled change, and verify the result. For this lesson, that means paying attention to symptom scope, recent changes, service state, logs, ports, resources, database reachability, and rollback options and keeping the command output as evidence.

Linux rewards precision: paths, users, permissions, units, and sockets all have exact meanings. When those details are treated casually, the common failure is making multiple untracked changes and losing the ability to identify the fix, which is harder to diagnose after several unrelated changes.

Also remember

- In an Odoo deployment, Odoo availability depends on systemd, Python, Odoo configuration, PostgreSQL, Nginx, disk,...
- Senior engineers build small repeatable checks around this area instead of relying on memory. They record the expected...

L18 Troubleshooting Checklist for Odoo Servers

Example and pitfalls

First five minutes of an Odoo outage

This captures time, service state, recent Odoo logs, listening ports, disk, and memory in a repeatable order. It avoids the common mistake of restarting first and destroying...

```
date -Is
sudo systemctl status odoo nginx postgresql --no-pager
sudo journalctl -u odoo --since "20 minutes ago" --no-pager
ss -ltnp | grep -E ':(80|443|8069|5432)\b'
df -h
free -h
```

Common mistakes

- Restarting every service before collecting evidence.
- Changing several variables at once during an incident.
- Ignoring recent deployments, package upgrades, or certificate renewals.
- Stopping after recovery without writing the prevention task.

Check database reachability as Odoo

```
sudo -u odoo -H bash -lc '
    source /opt/odoo/venv/bin/activate
    python - <<PY
import psycopg2
conn = psycopg2.connect(dbname="postgres", user="odoo", host="localhost")
print("database connection ok")
conn.close()
# ...
```

L18 Troubleshooting Checklist for Odoo Servers

Extra patterns

Incident order

- State the symptom and time window.
- Collect systemd, logs, ports, disk, memory, and database facts.
- Form one hypothesis and test it.
- Apply one reversible change.

Σ You should now be able to...

Section wrap-up

- L01 OS Fundamentals and Kernel Basics
- L02 Terminal Navigation with `pwd`, `cd`, `ls`, and `tree`
- L03 File Manipulation: Create, Copy, Move, Link, and Remove
- L04 Permissions with `chmod`, `chown`, and `chgrp`
- L05 Users, Groups, and `sudo`
- L06 Processes with `top`, `htop`, `ps`, and `kill`
- L07 `systemd` Services
- L08 `journalctl` and Log Files
- Complete every lab before the next section
- Keep a command/error journal
- Prefer disposable databases for experiments
- Re-read pitfalls before production changes

Ubuntu LTS

CLI

`systemd`

SSH

NEXT STEP

Complete the section labs

Open the next presentation deck

Section 01 · Linux

